

CH HIVE

- ☐ Introduction
- ☐ Architecture
- ☐ Type de données
- ☐ Modèle de données
- ☐ Bases de données
- ☐ Les tables
- ☐ Manipulation des données
- ☐ Requêtes Hive





- Hadoop est devenu un outil incontournable pour le traitement des données massives (Entrepôt de données, stockage de données d'entreprise, ...)
- Problème : MapReduce est un langage de très bas niveau → c'est-à-dire très proche du matériel → un développeur doit savoir interagir avec le cluster hadoop → pas aussi simple surtout pour les utilisateurs métier.
 - ➔ Langage d'abstraction proche de l'utilisateur et qui exprime des besoins métiers de la façon la plus simple possible
- **Hive** : est un langage d'abstraction
 - * conçus pour des utilisateurs métier (sans avoir besoin de connaître le bas niveau du matériels : cluster Hadoop)
 - * permet d'exprimer des jobs MapReduce d'une façon similaire au langage de requête SQL (très familière pour les utilisateurs métier de gestion des bases de données)
 - * Conçu originellement par FaceBook pour ses analystes et scientifiques de données pour exploiter les volumes de données énorme



Niveau d'abstraction

Langage complètement abstrait
proche de l'utilisateur. Conçu
pour les utilisateurs métier

Plutôt proche de l'utilisateur que
de la machine

Plutôt proche à la machine que
l'utilisateur

Langage bas niveau très proche
du matériel



SQL

HiveQL

Pig Latin

Hadoop (MapReduce)

Introduction



Word count en Java

```
package org.myc;

import java.io.IOException;
import java.util.*;

import org.apache.hadoop.fs.Path;
import org.apache.hadoop.conf.*;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;

public class WordCount {

    public static class Map extends Mapper<LongWritable, Text, Text,
    IntWritable> {
        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();

        public void map(LongWritable key, Text value, Context context) throws
        IOException, InterruptedException {
            String line = value.toString();
            StringTokenizer tokenizer = new StringTokenizer(line);
            while (tokenizer.hasMoreTokens()) {
                word.set(tokenizer.nextToken());
                context.write(word, one);
            }
        }
    }
}
```

```
public static class Reduce extends Reducer<Text, IntWritable, Text,
IntWritable> {

    public void reduce(Text key, Iterable<IntWritable> values, Context
context)
        throws IOException, InterruptedException {
        int sum = 0;
        for (IntWritable val : values) {
            sum += val.get();
        }
        context.write(key, new IntWritable(sum));
    }
}

public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();

    Job job = new Job(conf, "wordcount");

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);

    job.setMapperClass(Map.class);
    job.setReducerClass(Reduce.class);

    job.setInputFormatClass(TextInputFormat.class);
    job.setOutputFormatClass(TextOutputFormat.class);

    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    job.waitForCompletion(true);
}
```



Word count en HiveQL

```
CREATE TABLE docs (line STRING);

LOAD DATA INPATH 'docs' OVERWRITE INTO TABLE docs;

CREATE TABLE word_counts AS
SELECT word, count(1) AS count FROM
    (SELECT explode(split(line, '\s')) AS word FROM docs)
GROUP BY word
ORDER BY word;
```



■ C'est quoi Hive

- Hive est une infrastructure de stockage de données basée sur Hadoop.
- Hive est conçu pour faciliter la synthèse des données, l'interrogation ponctuelle et l'analyse de grands volumes de données.
- Hive fournit un langage de requête simple appelé HiveQL, qui est basé sur SQL et qui permet aux utilisateurs familiers avec SQL de faire ad-hoc interroger, résumer et analyser les données facilement.
- HiveQL permet aux utilisateurs d'intégrer des **Plug-in Mapreduce** pour être en mesure de brancher leurs mappeurs et réducteurs personnalisés pour faire une analyse plus sophistiquée qui peut ne pas être soutenu par les capacités intégrées du langage HiveQL.



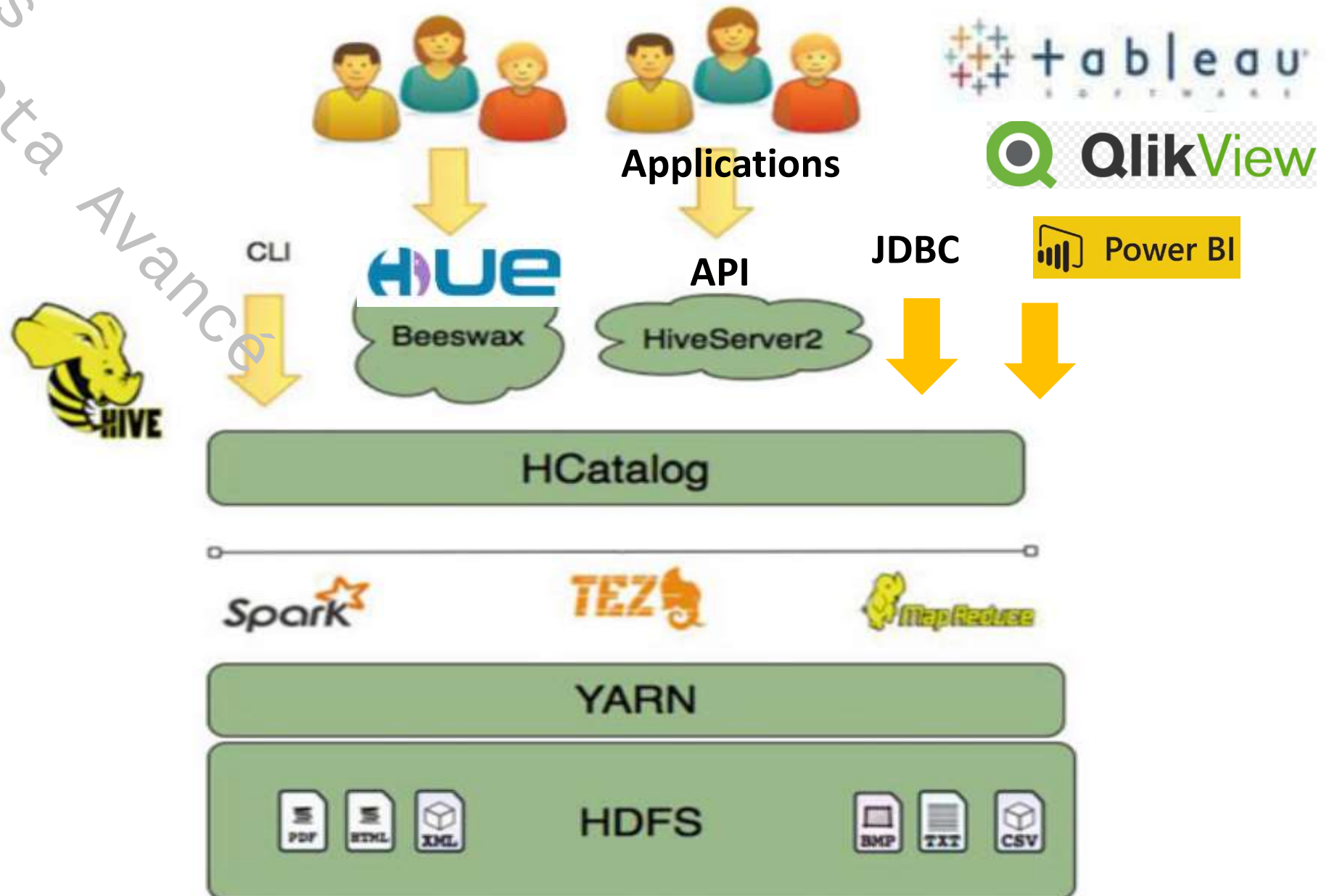
■ Ce que Hive n'est pas adéquat à faire

- Hadoop est un système de traitement par lots et les travaux Hadoop ont tendance à avoir une **latence élevée**
- Par conséquent, la latence des requêtes Hive est généralement très élevée (minutes) même pour les faibles tailles de données (centaines de mégaoctets) → En conséquence, Hive ne peut pas être comparé avec des systèmes tels qu'Oracle lorsque les analyses sont effectuées sur une quantité de données beaucoup plus faible.
- Hive **vis** à **fournir une latence acceptable** (mais pas optimal) pour la navigation interactive des données
- Hive n'est pas conçu pour le **traitement des transactions** en ligne et n'offre **pas** de requêtes en **temps réel** ni de mises à jour au niveau en ligne.
- Hive est adéquat pour les **travaux par lots** sur de grands ensembles de données immuables (ex. journaux web).

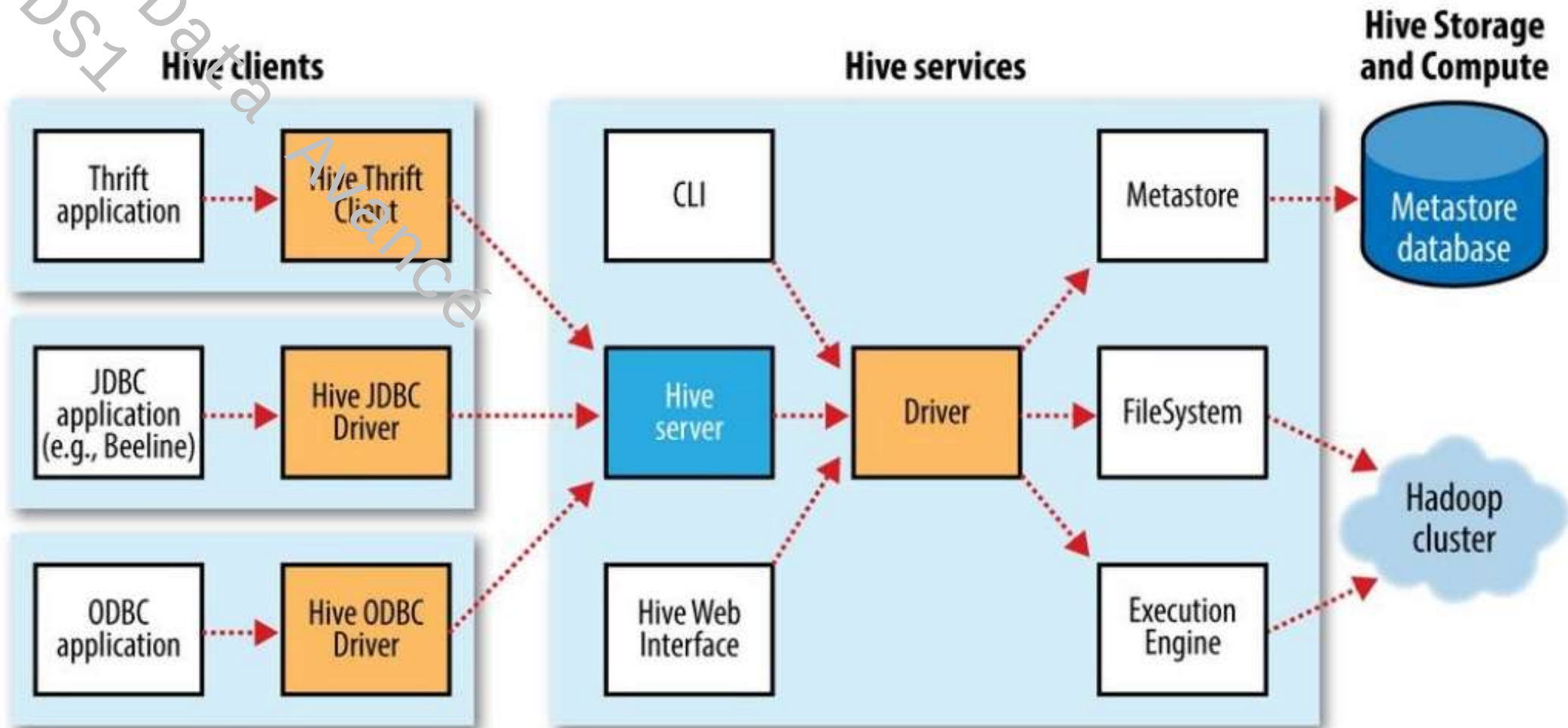


- Hive n'est pas un outil autonome. Il s'appuie sur divers composants pour stocker et interroger les données.
- Dans Hadoop écosystème, Hive est considéré comme un outil client d'accès aux données.
- L'accès aux données nécessite des ressources : **calcul, stockage, gestion**
- Hive est semblable à SQL mais ce n'est pas SQL, surtout en ce qui concerne la vitesse de traitement.
- Apache Hive traduit les programmes rédigés en langage HiveQL (SQL-like) en une ou plusieurs tâches Java **MapReduce, Tez ou Spark** (trois moteurs d'exécution pouvant être lancés sur Hadoop YARN).
- Mapreduce, Tez et Spark sont des traitements par lots alors que SQL est un langage de traitement interactif

Hive : Architecture



Hive : Architecture





■ Hcatalog

- Hcatalog : généralement considéré comme un composant séparé de Hive
- Hcatalog et Hive sont inséparables.
- Lorsque on crée une table Hive, on crée une structure dans Hcatalog
- Hcatalog facilite le partage de schémas entre les différents composants Hadoop. Hcatalog fournit un certain nombre de clés
- avantages :
 - ✓ Fournit un environnement de **schéma commun** pour plusieurs outils
 - ✓ Permet aux connecteurs des outils de lire les données et de les écrire dans l'entrepôt de Hive
 - ✓ Permet aux utilisateurs de **partager des données entre les outils**
 - ✓ **Crée une structure relationnelle** pour les données Hadoop
 - ✓ **Abstraction** de la gestion des données (accès/emplacement)



- L'infrastructure logicielle Hadoop est conçu pour stocker des données de divers natures (structurées, semi-structurées, non structurées) → **pas de schéma** de lecture au contrario des bases relationnelles où les données sont structurées dans des **tables** (lignes et colonnes)
- → Hive s'appuie sur une couche logicielle de stockage de données installée au dessous du moteur d'exécution (MapReduce, Tez, Spark) à l'exemple d'Hcatalog.
 - Non pas pour le stockage des données du HDFS
 - **MAIS** pour le stockage des **métadonnées** des requêtes des utilisateurs.
 - fait juste référence de pointeurs aux données qui sont sur le HDFS.
- Avant de lancer des requêtes HiveQL, une table (schéma de lecture) doit être crée dans la base de métadonnées (Metastore). Les tables HCatalog sont des **tables relationnelles** → abstraction de la lecture/écriture HDFS → Un langage semblable à SQL peut être utilisée (SQL-Like) → HiveQL

Hive : Architecture - Hiveserver2



- Les applications logicielles sont écrites via un langage de programmation spécifique (Python, Java, C#, ...)
- **OR** les bases de données ne sont accessibles que via des langages de requête (ex. SQL).
- **Problème** : connexion entre application logicielle et base de données
- **Solution** : interface de traduction entre les deux langages (application et base de données)
 - ✓ ODBC (Open Database Connectivity)
 - ✓ JDBC (Java Database Connectivity)
 - ✓ Thrift client (appeler un morceau de code dans un processus distant)
- ODBC : ne dépend pas d'un langage de programmation spécifique, d'un système de base de données ou d'un système d'exploitation
- JDBC : est une API pour le langage JAVA. C'est une interface qui permet à un client d'accéder à SGBD. JDBC est plus adapté pour les bases de données orientées objet. On peut accéder à n'importe quelle base de données compatible ODBC en utilisant le pont JDBC-to-ODBC.

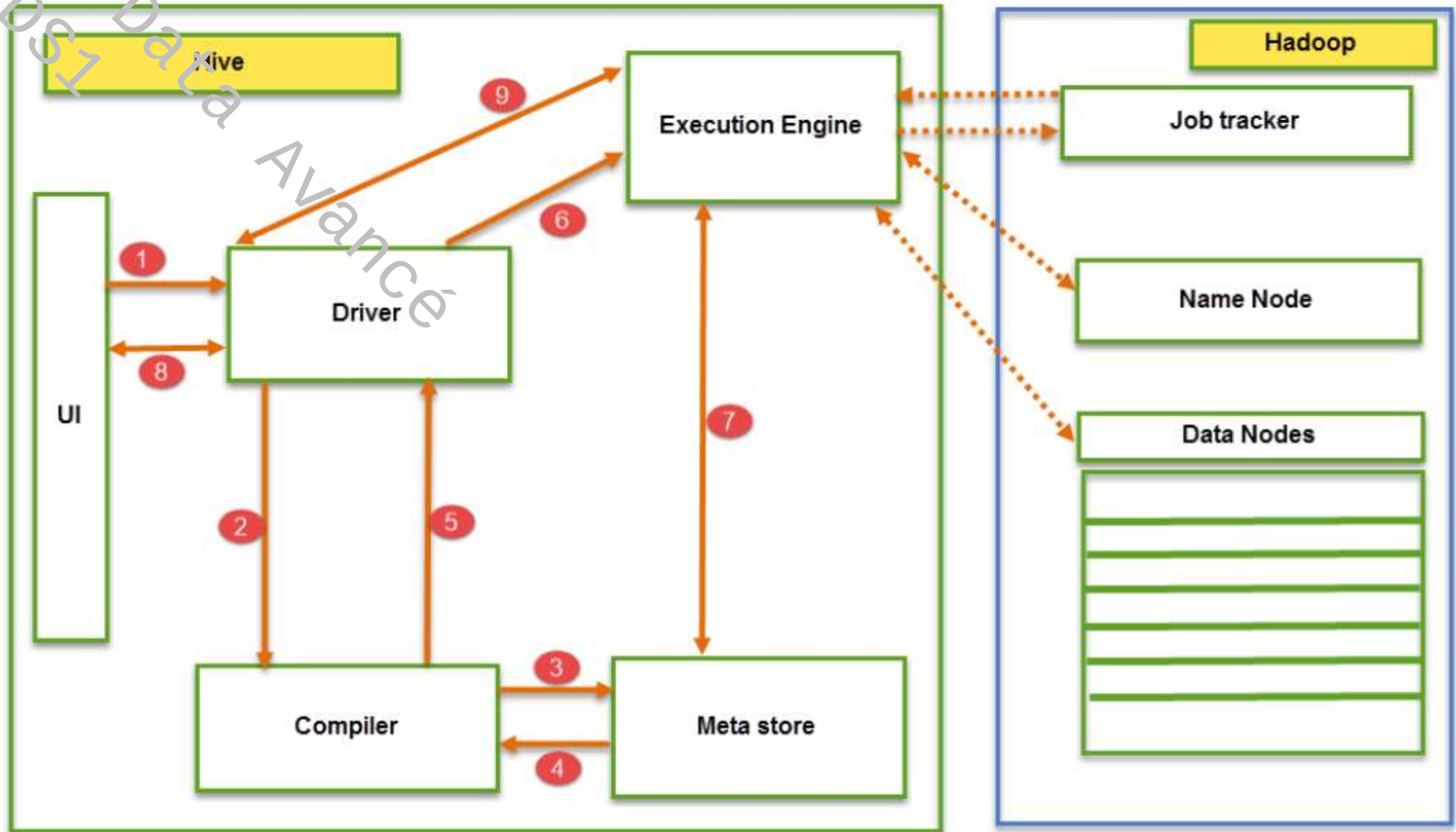


- Il existe plusieurs façons d'interagir avec Hive.
 - CLI : interface en ligne de commande.
 - Interface graphiques : plusieurs projets (commerciales / open source)
 - * Ambari
 - * Karmasphere
 - * Cloudera open source Hue (<https://github.com/cloudera/hue>),
 - * Qubole (<http://qubole.com>) et d'autres.
 - Thrift, JDBC, ODBC
 - Les API HiveServer qui permettent aux applications d'interagir avec Hive en utilisant des appels de procédure ou requêtes REST.
 - Les outils de Business Intelligence (BI) et de visualisation de données (Tableau, QlikView, PowerBI,...)



- Driver : c'est le composant qui gère les requêtes. Il est chargé d'orchestrer le cycle de vie d'une requête en invoquant les composants nécessaires dans le bon ordre
- Driver gère les sessions et garde une trace de statistiques.
- Driver : trois composants principaux
 - ✓ *Compiler* : vérifie l'exactitude syntaxique et sémantique des requêtes, puis il génère un plan physique sous la forme de DAG.
 - ✓ *Optimizer* : optimise le plan généré par le compilateur.
 - ✓ *Executor* : exécute les requêtes sur le cluster Hadoop. C'est l'interface entre le cluster Hadoop et Hive

Hive : Architecture - Fonctionnement





- Etape 1 : **executeQuery** : l'utilisateur utilise une interface (CLI, web, Thrift) pour lancer une requête
- Etape 2 : **getPlan** : *Driver* accepte la requête, crée un gestionnaire de session pour la requête et transmet la requête au compilateur pour générer le plan d'exécution.
- Etape 3 : **getMetaData** : Le compilateur demande au *MetaStore* de l'envoyer les métadonnées
- Etape 4 : **sendMetaData** : Le *MetaStore* envoie les métadonnées au compilateur. Le compilateur utilise ces métadonnées pour effectuer la vérification de type et l'analyse sémantique. Le compilateur génère ensuite le plan d'exécution **DAG** (Directed acyclic Graph).
- Etape 5 : **sendPlan** : Le compilateur envoie ensuite le plan d'exécution généré au *Driver*.
- Etape 6 : **executePlan** : Après avoir reçu le plan d'exécution du compilateur, *Driver* envoie le plan d'exécution au moteur d'exécution pour exécuter le plan.

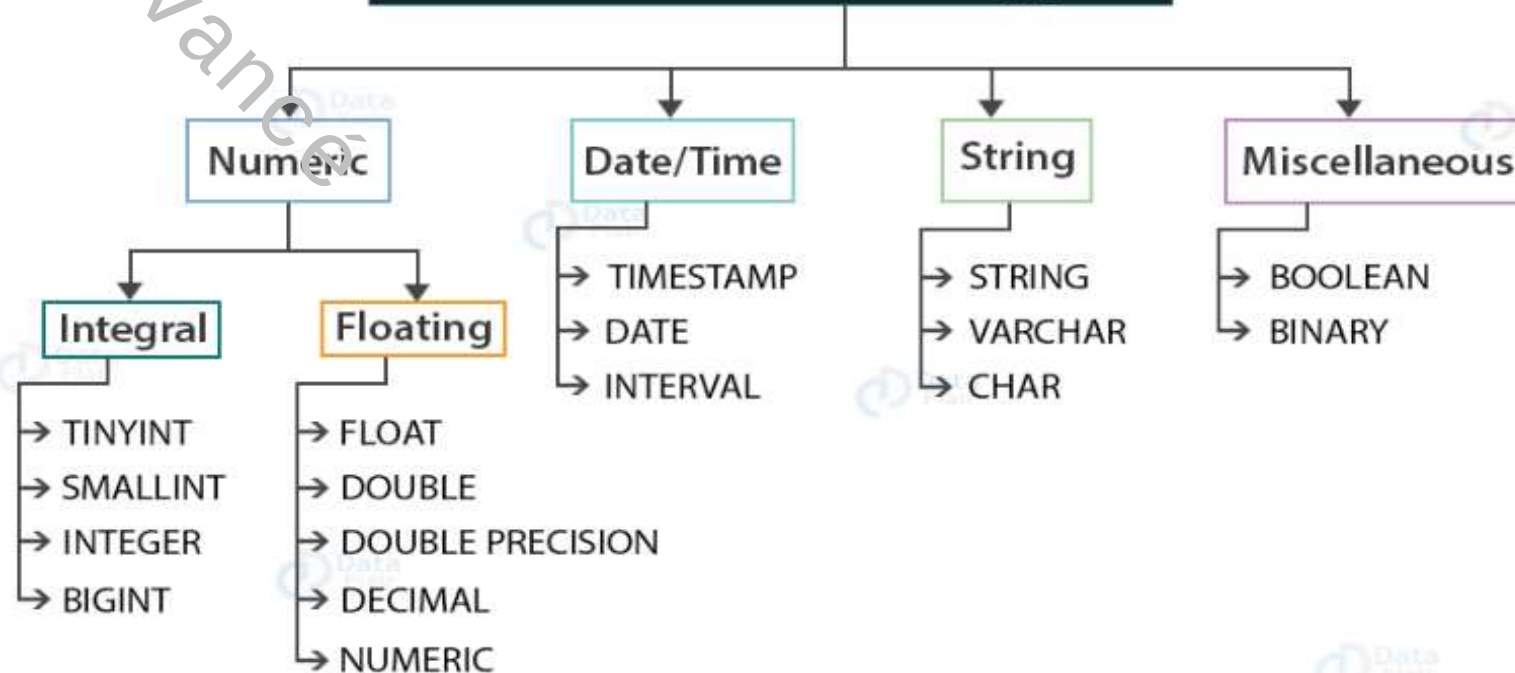


- Etape 7 : **Execution Engine (EE)** sert de pont entre Hive et Hadoop pour traiter la requête
 - ✓ EE doit d'abord contacter *NameNode*, puis les *DataNodes* pour accéder aux données selon les tables définies dans le *MetaStore* (EE récupère les données souhaitées à partir des *DataNodes*. Les données réelles des tables résident uniquement dans les *DataNodes*. À partir de *NameNode*, il ne récupère que les métadonnées pour la requête.
 - ✓ EE recueille par la suite les données réelles liées à la requête HiveQL à partir des *DataNodes*
 - ✓ EE communique d'une façon bidirectionnelle avec *MetaStore* présent dans Hive pour effectuer des opérations DDL (Data Definition Language. E.x. CREATE, DROP et ALTERING...).
 - ✓ EE communique à son tour avec les démons Hadoop tels que *NameNode*, *Data nodes* et *jobtracker* pour exécuter la requête sur le système de fichiers Hadoop
- Etape 8 : Récupération des résultats Driver
- Etape 9 : Envoi des résultats au EE



- Les types de données dans Hive spécifient le type de colonne/champ dans la table Hive → spécifie le type de valeurs qui peuvent être insérées dans la colonne spécifiée

Primitive Data Types



Complex Data Types



Hive : Type de données



Numeric Types:

Type	Memory allocation
TINY INT	1-byte signed integer (-128 to 127)
SMALL INT	2-byte signed integer (-32768 to 32767)
INT	4 -byte signed integer (-2,147,484,648 to 2,147,484,647)
BIG INT	8 byte signed integer
FLOAT	4 - byte single precision floating point number
DOUBLE	8- byte double precision floating point number
DECIMAL	We can define precision and scale in this Type

String Types:

Type	Length
CHAR	255
VARCHAR	1 to 65355
STRING	We can define length here(No Limit)

Miscellaneous

✓ BOOLEAN

True

False

✓ BINARY

tableau d'octets

Timestamp : Horodatage

Prend en charge l'horodatage

Unix traditionnel avec une précision nanoseconde en option

Date : sous format "YY-MM-DD"

Ex. 2020-04-22 11:22:30



ARRAY : Séquences ordonnées du même type qui sont indexés en commençant par zéro.

Ex. classe ARRAY<'MPDS1', 'Première Année'>
classe[0] → MPDS1

MAP : collection des paires clé/valeur. Accessible par clé

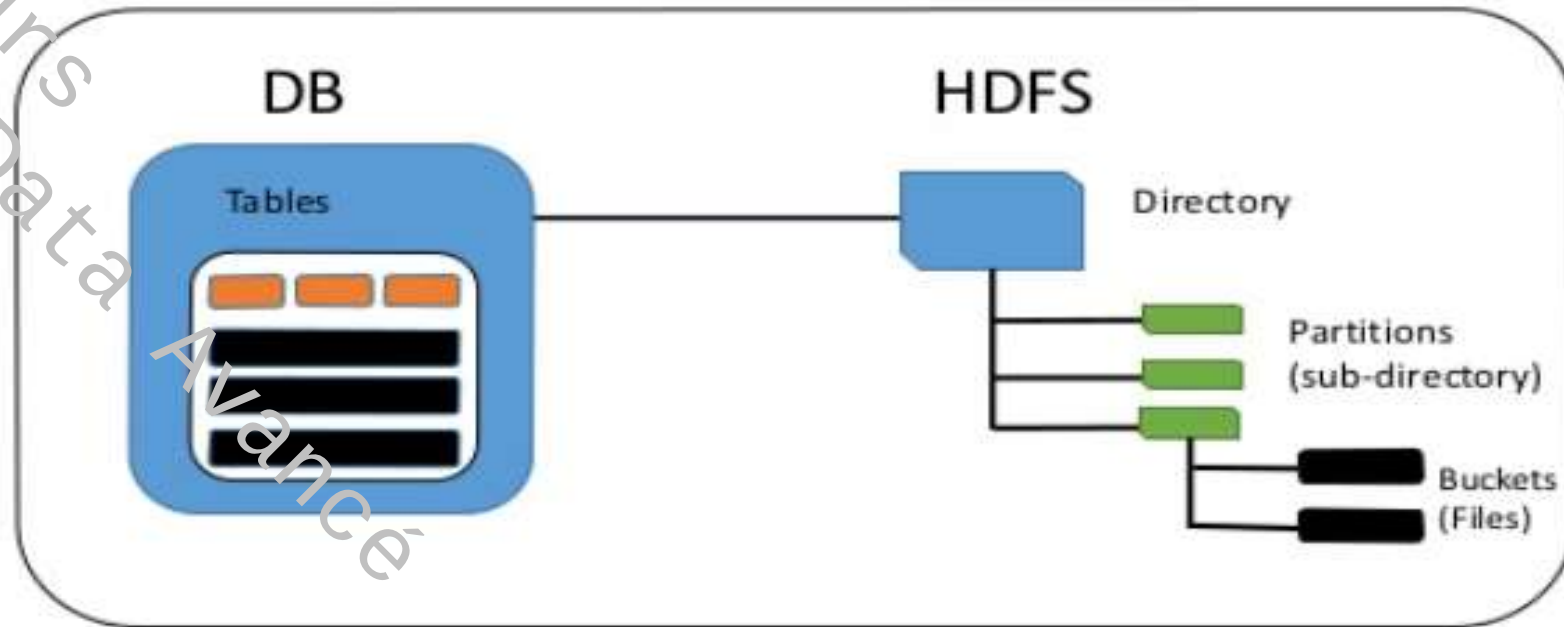
Ex. details map<'institution','FSB', 'niveau', 'mastère', 'spécialité', 'Data Science' >
details('institution') → 'FSB'

STRUCT : similaire à la structure dans C. Une structure est un objet qui contient divers champs qui peuvent être de n'importe quel type de données au contrario de ARRAY (même type)

EX. adresse STRUCT <Governorat:STRING, ville:STRING, codeZIP:INT>
adresse <'Bizerte', 'Zarzouna', 7021>
adresse.ville → 'Zarzouna'



- Deux considérations à prendre en compte
 - **Espace de stockage** : Hive est un système d'entrepôt de données open source construit au-dessus de Hadoop pour interroger et analyser les grands ensembles de données stockés dans les fichiers Hadoop.
 - **Types de données** : différents types
 - Structurées
 - Semi-structurées
 - Non-structurées
- ➔ Modèle de données (**structuration** & **accès**) :
- un **schéma** de
 - tableaux (colonnes, lignes),
 - partitions
 - saut
 - Ces objets sont des unités logiques définies dans la couche de métadonnées (Hive Metastore).
 - Le schéma Hive fait de sorte que les données sont vu en 02 dimensions comme de familières tables SQL indépendamment de leurs structures



- DB : Base de données
 - Le concept Hive d'une base de données n'est essentiellement qu'un catalogue (espace de noms de tables)
 - Utiles pour les grands clusters avec plusieurs équipes et utilisateurs → un moyen pour éviter les collisions de noms de table.
 - permet d'organiser des tables de production en groupes logiques.



- Si on spécifie pas la base, le répertoire de la base par défaut est utilisée (default)
- Hive crée un répertoire pour chaque base
- Création : **DATABASE** ou **SCHEMA**
hive> **CREATE DATABASE mpds1;**
- Lister les bases de données
hive> **SHOW DATABASES;**
default
fsb
mpds1
mrsi1
- Chercher une base (expression régulières)
hive> **SHOW DATABASES LIKE 'f.*';**
fsb
hive> ...



- Le répertoire de la base de données est créé sous un répertoire de premier niveau défini par la propriété : **hive.metastore.warehouse.dir.**
- Valeur par défaut : **/user/hive/warehouse**
- Ex. la base fsb est créée dans **/user/hive/warehouse/fsb.db**
- Il est possible de choisir un autre répertoire

```
hive> CREATE DATABASE fsb  
      > LOCATION '/mon/repertoire/prefere';
```

- Ajouter une description (commentaire) :

```
hive> CREATE DATABASE mpds1  
      > COMMENT 'suivie d'enseignement à distance de MPDS1';
```

```
hive> DESCRIBE DATABASE mpds1;  
mpds1 suivie d'enseignement à distance de MPDS1  
hdfs://master-server/user/hive/warehouse/mpds1.db
```

RQ : Une instruction se termine par « ; » → plusieurs lignes pour une seule instruction



- Plus d'information : clé-valeurs
- Un moyen d'ajouter des informations à la sortie
- DESCRIBE avec l'argument EXTENDED

```
hive> CREATE DATABASE mpds1  
      > WITH DBPROPERTIES ('niveau' = première année, 'date' =  
'2020-06-08');
```

```
hive> DESCRIBE DATABASE mpds;  
mpds hdfs://master-server/user/hive/warehouse/mpds.db
```

```
hive> DESCRIBE DATABASE EXTENDED mpds;  
mpdss hdfs://master-server/user/hive/warehouse/financials.db  
{date=2020-06-08, niveau=première année};
```



- Changer le répertoire courant

hive> **USE** fsb;

→ Maintenant, les commandes telles que SHOW TABLES ; afficheront la liste des tables dans cette base de données.

- supprimer une base

hive> **DROP DATABASE IF EXISTS** fsb;

L'option IF EXISTS est facultative et supprime les avertissements si la base de données fsb n'existent pas.

- Par défaut, Hive ne permet pas de supprimer une base de données si elle contient des tables. On doit tout d'abord supprimer les tables en premier ou ajouter le mot-clé CASCADE à la commande (Hive supprime les tables dans la base de données en premier)

hive> **DROP DATABASE IF EXISTS** fsb **CASCADE**;



■ Tables

- Le modèle de données Hive permet une vue **logique** en ligne/colonne des données appelée une table.
- Semblable à une base relationnelle, une table Hive consiste en une définition bidimensionnelle des données.
- Cependant, les données existent indépendamment de la table.
- Les données d'une table Hive existent dans un répertoire HDFS et la définition de la table (métadonnée) est stockée dans une base de données relationnelle (Hcatalog).

■ SGBD Vs Hive

- Dans les bases de données relationnelles, **la suppression d'une table supprime sa définition et les données sous-jacentes.**
- Dans Hive, si on définit une table comme une table externe, la définition de la table sera indépendante des données → **la suppression de la table n'entraîne pas la suppression des données**
- les mêmes données peuvent être associées à multiples tables
- Hive → plusieurs types de données (contrairement à SGBD)



- Hive a deux types de tables :
 - Internal or Managed** Tables (Table gérée)
 - External** Table (externe)
- External Table
 - Les tables externes sont créés à l'aide de mots-clés EXTERNES avec l'instruction CREATE TABLE.
 - Il s'agit de type de table recommandé pour tous les déploiements de production de Hadoop
 - Dans le cas de tables externes, Hive ne supprime pas les données du système de fichiers comme si la table est supprimée
- Internal Table
 - Une table interne dans Hive se réfère à une table dont les données sont **gérées** par Hive.
 - Si on supprime une table interne → Hive supprimera également ses données sous-jacentes
 - Ces tables ne sont pas utilisés très souvent dans Hadoop



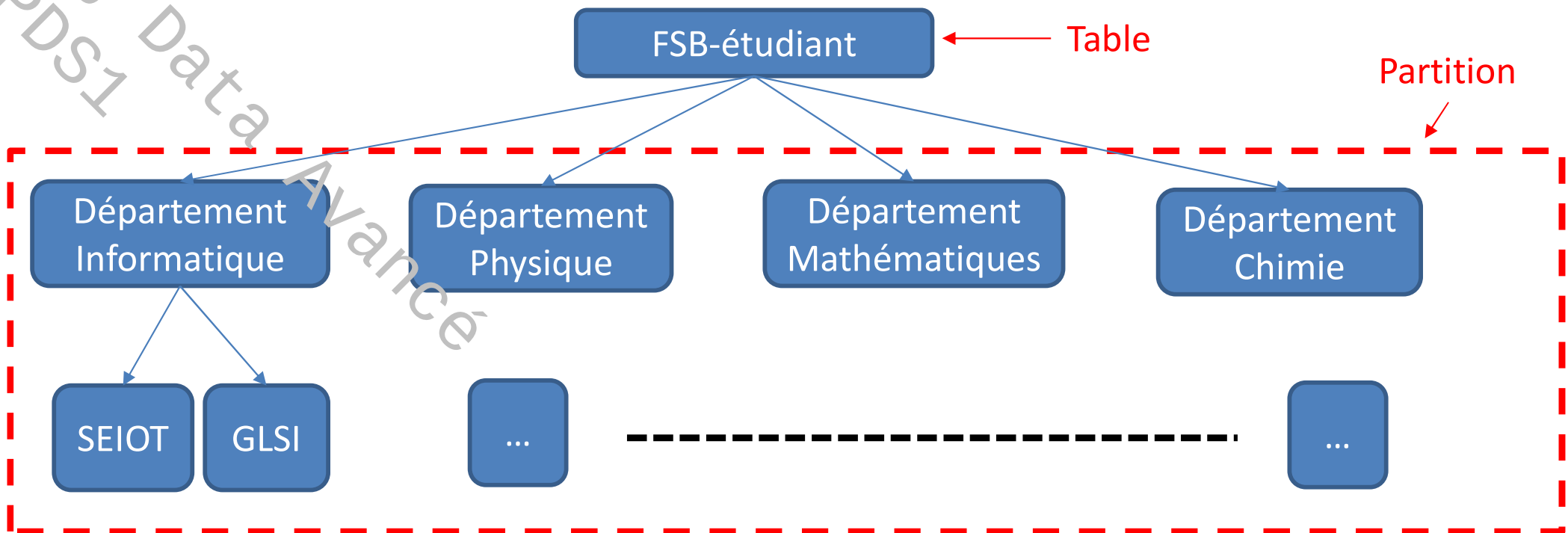
■ Partition

- Utilisé pour distribuer la charge horizontalement en déplaçant les données physiquement le plus proche possible des ses utilisateurs les plus fréquents.
- Hive a la notion de partition des tables
 - ✓ Performance
 - ✓ Organisation logique des données (Hiérarchique)

■ Ex.

- Reprenons l'exemple de faculté des sciences de Bizerte
- Nos administrateurs lancent des requêtes avec filtrage (WHERE) pour sélectionner des étudiants particuliers : département, section, niveau, ..

```
hive> CREATE TABLE fsb (  
  > nom STRING, age INTEGER, genre STRING  
  > departement STRING, section STRING, niveau INTEGER  
  > adresse STRUCT<rue:STRING, ville:STRING, zip: INTEGER  
  > )  
  > PARTITIONED BY (departement STRING, section STRING);
```



- La partition des tables change la façon dont Hive structure le stockage des données.
- Supposons que la table est créée dans la base *madb*
`hdfs://master_server/user/hive/warehouse/madb.db/fsb`
→ Hive va maintenant créer des sous-répertoires reflétant la structure de partitionnement.



- Répertoires hiérarchiques créés
 - .../fsb/departement=informatique/section=SEIOT
 - .../fsb/departement=informatique/section=GLSI
 - ...
 - .../fsb/departement=chimie/section=chimie industrielle
- Chaque répertoire & sous répertoire contient 0 fichier ou plus
- Une fois la table est créée, les clés de partition se comportent comme des colonnes d'une base relationnelle.
- Ex.

```
SELECT * FROM fsb
```

```
WHERE departement = 'Informatique' AND section = 'SI';
```

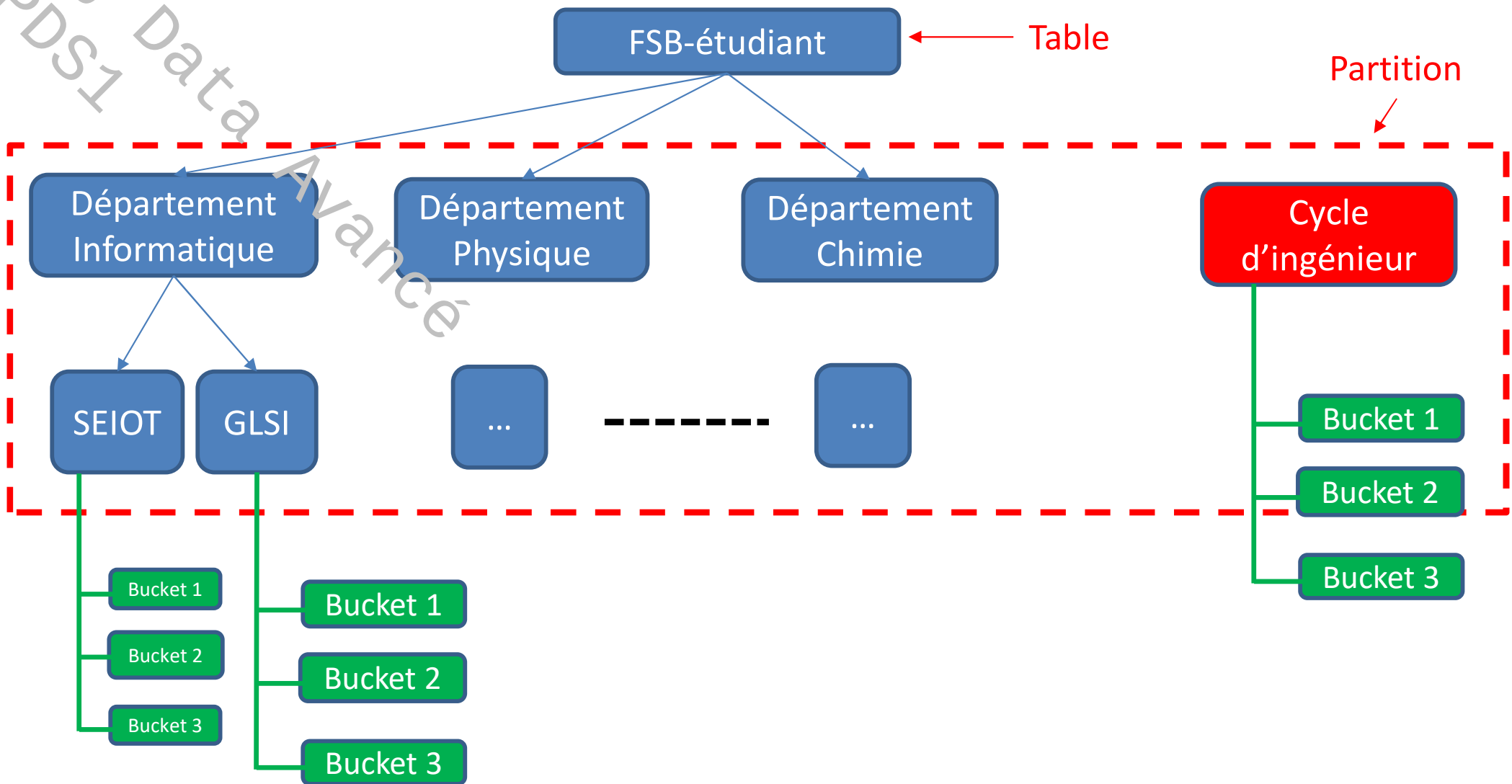
→ sélectionner tous les étudiants de la section SI du département informatique

Remarque : En réalité, on aurait pu lancer la même requête sans passer au préalable par la partition. Toutefois, pour des larges bases de données ce qui est le cas de Big Data ça prendra beaucoup de temps → intérêt de partition



► Saut (Bucket)

- C'est un outil supplémentaire de structuration des données
- but : améliorer les performances des requêtes HiveQL
- Permet de subdiviser les tables et/ou les partitions
- Solution au problème de sur-partitionnement et qui permet de décomposer les données en ensembles de fichier plus gérables
- Sur-partitionnement :
 - Reprenons l'exemple de la table fsb.
 - supposons qu'on vient de créer un nouveau département (cycle d'ingénieur en Sciences des données). Le nombre des étudiants sera très limité et par conséquent la taille du fichier relatif à ce nouveau département sera probablement petite par rapport au autres départements.
 - ➔ pour des raisons de performances des requêtes (temps de réponse), il vaut mieux avoir des fichiers de tailles équilibrées.
 - ➔ **Bucket** donne la main à l'utilisateur de subdiviser les fichiers de grande taille en plusieurs fichiers de taille plus petites





- Il faut fixer la variable booléenne (hive.enforcing.bucketing= TRUE) à TRUE
- Les buckets sont créés grâce à une fonction de Hachage qui utilise une clé de bucket pour créer les buckets.
- Il faut choisir judicieusement la clé et éviter le problème de **Skewness** relatif à la distribution des données (choisir la clé avec grand nombre de valeurs distinctes) (*prendre note du tableau*)
- Ex. table fsb

```
hive> CREATE TABLE fsb (  
  > name STRING, age INTEGER, genre STRING  
  > departement STRING, section STRING, niveau INTEGER  
  > adresse STRUCT<rue:STRING, ville:STRING, zip: INTEGER  
  > )  
  > PARTITIONED BY (departement STRING, section STRING);  
  > CLUSTERED BY (age) INTO 3 BUCKETS;
```



■ Création

- Instruction **CREATE TABLE**.
- La version de CREATE TABLE de Hive est assez similaire à SQL standard. Cependant, elle fournit diverses options pour ajouter à la polyvalence de la gestion de divers types trouvés dans le monde Big Data.
- Les données sont de différents types (pas toutes structurées en lignes-colonnes) → Configuration spécifiée lors de la création d'une table qui définit comment Hive interprétera les données
- Hive possède **interpréteurs de données** natifs (**built-in**) qui permettent à Hive d'interpréter plusieurs types de données (les plus courants) : appelées **Serdes** (Serialisation Deserialisation)
- Hive permet également de définir des propres **Serdes** et il suffit de les brancher dans une instruction CREATE TABLE pour permettre à Hive de comprendre le format des données.



Exemple

```
CREATE TABLE IF NOT EXISTS mydb.MPDS1 (  
  NomPrenom STRING COMMENT 'nom et prénom de l'étudiant' ,  
  moyenne FLOAT COMMENT 'la moyenne' ,  
  matieres ARRAY<STRING> COMMENT 'liste de matières' ,  
  moyMatiere MAP<key:STRING, values:FLOAT> COMMENT 'key : la matière, values : moyenne par matière' ,  
  adresse STRUCT<rue:STRING, ville:STRING, gouvernorat:STRING, zip:INT>  
  COMMENT 'Home address' )  
  
COMMENT 'Description de la table MDS1'  
TBLPROPERTIES ('créateur'='moi', 'créé_at'='2020-06-10 10:00:00', ...)  
  
LOCATION '/user/hive/warehouse/mydb.db/MPDS1' ;
```



- IF NOT EXISTS** : Par défaut Hive vérifie une table de même nom existe au moment de la création. Si oui, Hive génère un « warning ». **IF NOT EXISTS** permet de ne pas générer un « warning » et de créer une table avec un nom existant en écrasant l'ancienne table (**Faites attention quant à son utilisation**)
- mydb.MPDS1 : On peut spécifier l'emplacement de la table en attachant la table créée à une base par (.)
 - On peut ajouter un commentaire à n'importe quelle colonne, après le type.
 - Comme il est le cas pour les bases de données, on peut joindre un commentaire à la table elle-même.
 - Le principal avantage de **TBLPROPERTIES** est d'ajouter de la documentation supplémentaire dans un format clé-valeur.
 - On peut spécifier optionnellement un emplacement pour les données de la table.



- Copier le schéma d'une table
 - Il est possible de copier le schéma (mais pas les données) d'une table existante :

```
CREATE TABLE IF NOT EXISTS mydb.MPDS2  
LIKE mydb.MPDS1;
```

- Lister les tables d'une base de données

- Base de travail courante :

```
hive> USE mydb;  
hive> SHOW TABLES;  
MPDS1  
MPDS2
```

- Base de travail différente

```
hive> USE default;  
hive> SHOW TABLES IN mydb;  
MPDS1  
MPDS2
```



- Expressions régulières
- Recherche d'une table

```
hive> USE mydb;
```

```
hive> SHOW TABLES;
```

```
MPDS1
```

```
MPDS2
```

```
SI1
```

```
SI2
```

```
SI3
```

```
hive> SHOW TABLES 'MPDS.*';
```

```
MPDS1
```

```
MPDS2
```



- Afficher les commentaires et détails de la table
- On utilise la commande **DESCRIBE EXTENDED** mydb.MPDS1 pour afficher des détails sur la table.
- On peut supprimer le préfixe *mydb*, si on utilise actuellement la base de données *mydb*

```
hive> DESCRIBE EXTENDED mydb.MPDS1;
```

- On peut afficher les commentaires d'une colonne

```
hive> DESCRIBE mydb.MPDS1.moyenne;
```

```
moyenne float la moyenne finale de l'étudiant
```



► Lecture des fichiers

- La déclaration suivante crée une table externe qui peut lire tous les fichiers de données délimités par des virgules dans **/data/stocks** (**table externe**):

```
CREATE EXTERNAL TABLE IF NOT EXISTS stocks (  
  exchange STRING,  
  symbol STRING,  
  ymd STRING,  
  price_open FLOAT,  
  price_high FLOAT,  
  price_low FLOAT,  
  price_close FLOAT,  
  volume INT,  
  price_adj_close FLOAT)  
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','  
LOCATION '/data/stocks';
```

- Parce qu'elle est externe, supprimer la table ne supprime pas les données, bien que les métadonnées de la table soient supprimées.



■ Supprimer une table

- La commande familière **DROP TABLE** de SQL est supportée :
DROP TABLE IF EXISTS MPDS1;
- Les mots-clés IF EXISTS sont facultatifs. Si ils ne sont pas utilisés et que la table n'existe pas, Hive retourne une erreur.
- Pour les tables gérées, les métadonnées et les données de la table sont supprimées.

■ Modifier la table

- La commande **ALTER TABLE** permet de modifier la table
- ALTER TABLE modifie uniquement les métadonnées de la table. Les données de la table sont intactes.
- On doit s'assurer que toutes les modifications sont cohérentes avec les données réelles.



- Ex. renommer la table

```
ALTER TABLE log_messages RENAME TO logmsgs;
```

- Ex. Ajouter des partitions

```
ALTER TABLE logmsgs ADD IF NOT EXISTS  
PARTITION (year = 2011, month = 1) LOCATION '/logs/2011/01'  
PARTITION (year = 2011, month = 2) LOCATION '/logs/2011/02'  
PARTITION (year = 2011, month = 3) LOCATION '/logs/2011/03';
```

- Ex. Eliminer des partitions

```
ALTER TABLE logmsgs DROP IF EXISTS PARTITION(year = 2011, month = 2);
```

- Ex. Ajouter une colonne

```
ALTER TABLE logmsgs ADD COLUMNS (  
app_name STRING COMMENT 'Application utilisée';
```

- *Remarque : Consulter le site officiel <http://hive.apache.org/>*
- *<https://cwiki.apache.org/confluence/display/Hive/Tutorial>*

Manipulation des données - Loading Data



- **Télécharger les données dans une table**
 - Puisque Hive n'a pas d'opérations d'insertion, de mise à jour et de suppression au niveau des lignes de la table, la seule façon de mettre les données dans un tableau est d'utiliser l'une des opérations de chargement.
 - Ou on peut simplement écrire les fichiers dans les bons répertoires par d'autres moyens (on peut copier un fichier à l'emplacement spécifié en utilisant les commandes HDFS **put** ou **copy** et créer un tableau pointant vers cet emplacement avec toutes les informations de format de ligne pertinentes.)
 - Exemple

```
LOAD DATA LOCAL INPATH '/data/MPDS1'
OVERWRITE INTO TABLE MPDS
PARTITION (niveau = 'première_année');
```
 - La commande crée une partition (si elle n'existe pas) et copie les données dans la partition créée
 - **LOAD DATA LOCAL** ... copie les données locales à l'emplacement final dans le système de fichiers distribué, tandis que **LOAD DATA** ... (c.-à-d. sans **LOCAL**) déplace les données jusqu'à l'emplacement final.



- Synthaxe

LOAD DATA [LOCAL] INPATH 'filepath' [OVERWRITE] INTO TABLE tablename

- **LOAD DATA** : mot clé pour télécharger les données dans Hive.
- **LOCAL** si utilisé, permet de téléchrager les données de repertoire local. Si omit, les données sont téléchargées depuis le repertoire spécifié par la variable **fs.default.name**
- **INPATH** 'filepath'
 - si LOCAL est utilisé: file:///user/hive/example
 - si LOCAL is omit : dfs://namenode:9000/user/hive/example
- **OVERWRITE** : téléchrage les données dans une table existante et écrase les données existantes. Si OVERWRITE est omit, les données existantes sont augmentées par les nouvelles.
- **INTO TABLE** : une table dans Hive



▪ Insertion via requêtes Hive

- Opération communément utilisée
- Insérer des données dans des tables (intermédiaires ou finales) à partir des requêtes Hive lancées sur des tables existantes et qui reflètent un traitement données

- **OVERWRITE** : remplacer les données existantes d'une tables par le résultat d'une requête

INSERT OVERWRITE TABLE etudiant2 **SELECT** id, nom, age, moyenne **from** etudiant1;

- **INTO** : Etendre la table existante

INSERT INTO TABLE etudiant2 **SELECT** id, nom, age, moyenne **from** etudiant1;



- Création d'une table et insertion de données (une seule requête)

```
CREATE TABLE MPDS1
```

```
AS SELECT nom, prenom, moyenne
```

```
FROM etudiant_informatique ei
```

```
WHERE ei.section = 'MPDS1';
```

RQ : le schéma de la table créée est défini par **AS**

- Insertion directe d'une ligne dans une table

```
INSERT INTO TABLE MPDS1 values('Salem', 'Salem', 17.28);
```

- Exporter les données d'une table Hive

```
INSERT OVERWRITE LOCAL DIRECTORY '/tmp/MPDS1_etudiant'
```

```
ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t'
```

```
SELECT * FROM MPDS1;
```



- Reprenons l'exemple du transparent 37
 - SELECT : opérateur de projection
 - hive> **SELECT** NomPrenom, moyenne **FROM** MPDS1;
Salem salem 12.42
Marnissi Khouloud 13.65
 - Avec alias
 - hive> **SELECT** m.NomPrenom, m.moyenne **FROM** MPDS1 m;
Salem salem 12.42
Marnissi Khouloud 13.65
 - Il est possible d'accéder un champ spécifique d'une structure complexe dans une table (*matieres* est un type complexe de données ARRAY ordonné → accessible par position)
 - hive> **SELECT** m.NomPrenom, m.matieres[0] **FROM** MPDS1 m;
Salem Salem BigData



- Sélection d'un champ spécifique d'une STRUCT (un champ de STRUCT est accessible par (.))
`hive> SELECT NomPrenom, adresse.ville FROM MPDS1;`
Salem Salem NouakChott
Saada Islem Bizerte
- Sélection d'un champ spécifique d'une structure MAP
`hive> SELECT NomPrenom, moyMatiere["BigData"] FROM MPDS1;`
Salem Salem 14.3
Saada Islem 11.5
- Spécifier des colonnes avec des expressions régulières
`moyenne (FLOAT) et moyMatiere (MAP)`
`hive> SELECT NomPrenom, `mo*` FROM MPDS1;`
Salem Salem 12,36 {BigData:14.3, DataViz:10.5, ML:13.6}
Saada Islem 12.24 {BigData:11.5, DataViz:13.5, ML:11.5}



- Opération et calcul avec les colonnes

- Fonction

```
hive> SELECT upper(NomPrenom), round(moyenne) FROM MPDS1;  
SALEM SALEM 12  
MARNISSI KHOULOU 14
```

- Calcul

```
hive> SELECT * FROM MPDS1 WHERE moyenne >= 12;
```

```
hive> SELECT * FROM MPDS1  
WHERE moyenne >= 12 && moyMatiere["BigData"] >10;
```




Fonctions

Round() → entier le plus proche

Floor() → partie entière

Ceil() → partie entière +1

exp(), ln(), log10(), log2(), ...

Power(x,n) → x^n

Sqrt()

Sin(), cos(), tan()

....

Opérateurs binaires (bit par bit)

A & B

A | B

A ^ B (XOR)

~A (NOT)

Opérateurs Arithmétiques

A + B

A - B

A * B

A / B

A % B (modulo)

Opérateurs relationnels

A = B

A != B

A > B, A >= B

A < B, A <= B

A is NULL, A is NOT NULL

A LIKE B

Opérateurs logique

A && B (AND)

A || B (OR)

!A (NOT)



■ Agrégation

- Un type spécial de fonction est la fonction d'agrégat qui renvoie une valeur unique résultant de certains calculs sur de nombreuses lignes.

- Ex.

```
hive> SELECT count(*) avg(moyenne) FROM MPDS1;
```

10 13.69

Nombre des
étudiant

La moyenne des moyennes des étudiants

Fonctions d'agrégation

Count(*), count(expression) → nombre de lignes (qui satisfont l'expression)

Count(DISTINCT col) → comptage des valeurs distinctes

Sum(col), avg(col), min(col), max(col), variance(col)

Corr(col1,col2),

....



- Condition : CASE ... WHEN ... THEN

```
hive> SELECT NomPrenom, moyenne,  
> CASE  
> WHEN moyenne < 10.0 THEN 'redouble'  
> WHEN moyenne >= 10.0 AND moyenne < 12.0 THEN 'Passable'  
> WHEN moyenne >= 12.0 AND moyenne < 14.0 THEN 'Assez Bien'  
> WHEN moyenne >= 14.0 AND moyenne < 16.0 THEN 'Bien'  
> ELSE 'Très Bien'  
> END  
> FROM MPDS1;
```

```
salem salem    12,36    Assez Bien  
Marnissi Khouloud  13.65  Assez Bien
```



■ Jointure

- Hive prend en charge les jointures d'égalité entre les tables pour permettre de combiner les données de deux tables.
- Syntaxe

SELECT table_fields

FROM table_one

JOIN table_two

ON (table_one.key_one = table_two.key_one

AND table_one.key_two = table_two.key_two);



- Exercice (exemple - Jointure) : Analyser le code suivant en identifiant le rôle de chaque bloc.

```
USE MPDS1;  
CREATE TABLE MPDS1.listeEtudiant (  
  CIN int,  
  nom string,  
  prenom string  
)  
CLUSTERED BY (CIN) INTO 1 BUCKETS
```

```
INSERT INTO TABLE MDPS1.listeEtudiant  
VALUES  
(05002536, 'salem', 'salem'),  
(15669854, 'Marnissi', 'Khouloud'),
```



```
CREATE TABLE MPDS1.adresse (  
  CIN INT  
  ad string  
)  
CLUSTERED BY (CIN) INTO 1 BUCKETS
```

```
INSERT INTO TABLE MPDS1.adress2  
VALUES  
(05002536, 'NouakChott'),  
(15669854, 'Bizerte'),
```

```
SELECT listeEtudiant.CIN,  
listeEtudiant.nom,  
listeEtudiant.prenom  
FROM  
MPDS1.listeEtudiant  
JOIN  
MPDS1.adresse  
ON (listeEtudiant.CIN = adresse.CIN);
```

Exercice : Exemple d'un script Hive



Exercice : Analyser le script Hive suivant en complétant les champs réservés aux commentaires afin de le rendre plus compréhensible

```
$HIVE_HOME/bin/hive
## .....
CREATE DATABASE census;
## .....
USE census;
## .....
CREATE TABLE person (
  persid int,
  lastname string,
  firstname string
)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ',';
## .....
LOAD DATA LOCAL INPATH 'file:///root/hive/example/person001' OVERWRITE INTO
TABLE person;
## .....
SELECT persid, lastname, firstname FROM person;
```